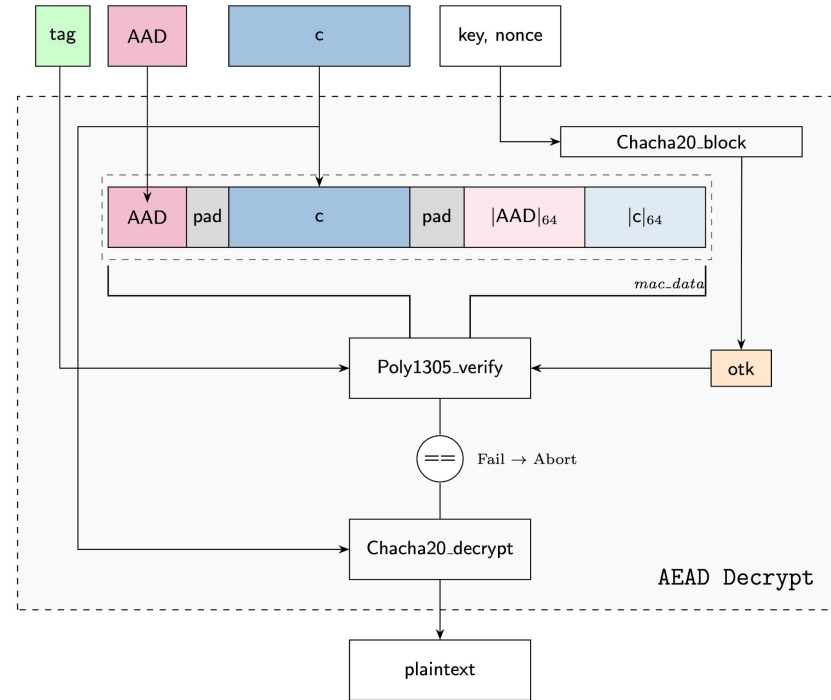
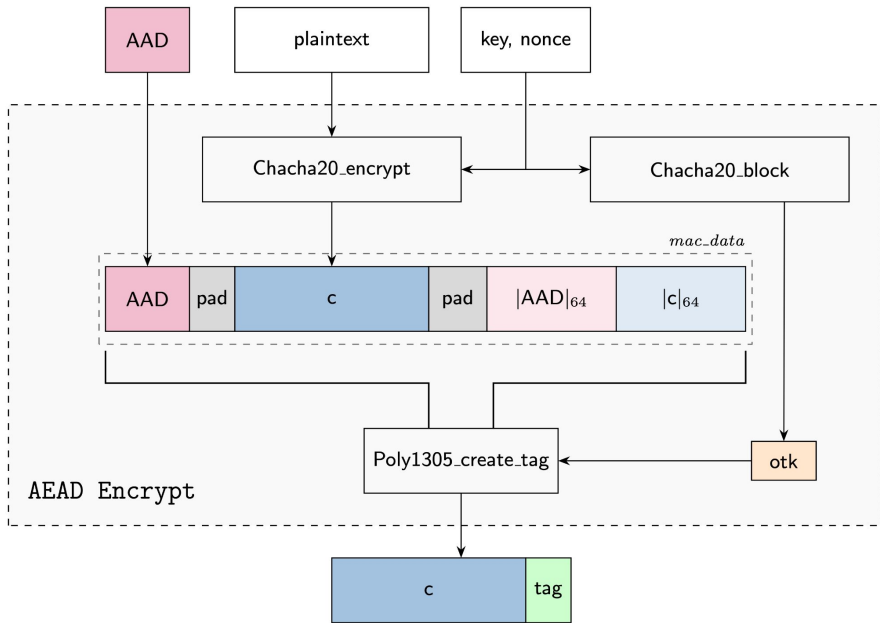


Chacha20-Poly1305/2133

Team 33

AEAD Scheme



Functions & Cost Analysis (with $\text{ptxt_len} = n$, $\text{aad_len} = m$)

- (1) `ChaCha_20_block(key, nonce) → poly_key`
 - $O(1962)$
- (2) `ChaCha_20_encrypt(ptxt, key, nonce, block_ctr) → ctxt`
 - $O(n/64 * 2221)$
- (3) `Poly1305_init(poly_key) → clamp r, s`
 - $O(200)$
- (4) `Create_tag(mac_data, r, s) → tag`
 - $O((n+m)/16 * 271)$

Total Cost Analysis on our first implemented functions:

$(1)+(2)+(3)+(4) = O(n)$ (considering $n \gg m$)

Processor Architecture

- Processor: Intel Core i7-7700HQ
- Kaby Lake architecture (4 ALU scalar Ports, 3 ALU vectorized ports)
- Instruction Set: 64-bit
 - Extensions : Intel® SSE4.1, Intel® SSE4.2, Intel® AVX2
- L1 Cache: 32 KB (per core)
- L2 Cache: 256 KB (per core)
- L3 Cache: 6 MB (shared)

ChaCha20 Scalar Optimizations

1. Strength reduction (avoid div, mod)
 - speedup approx. 1.17x (-O3), 1.09x (-O0)
2. Inlining
 - speedup approx. 1.46x (-O3), 2.04x (-O0)
3. Scalar Replacement
 - replace working state with scalar variables
 - avoid redundant reads and writes
 - speedup approx. 1.52x (-O3), 2.23x (-O0)

ChaCha20 - Unrolling + ILP

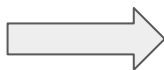
1. Unroll loop + ILP (handle multiple plaintexts at once)
 2. Unroll loop + ILP (ciphertext computation)
- Speedup:
 - 1.50x with -O3, 2.38x with -O0 (4 blocks at once)
 - 1.27x with -O3, 2.38x with -O0 (8 blocks at once)
 - Reasoning (Speedup):
 - Register Spilling, O3 does a lot on its own and gets backed into corner
 - Ilp is maximized already with 4 blocks (with -O0)

ChaCha20 - Vectorization



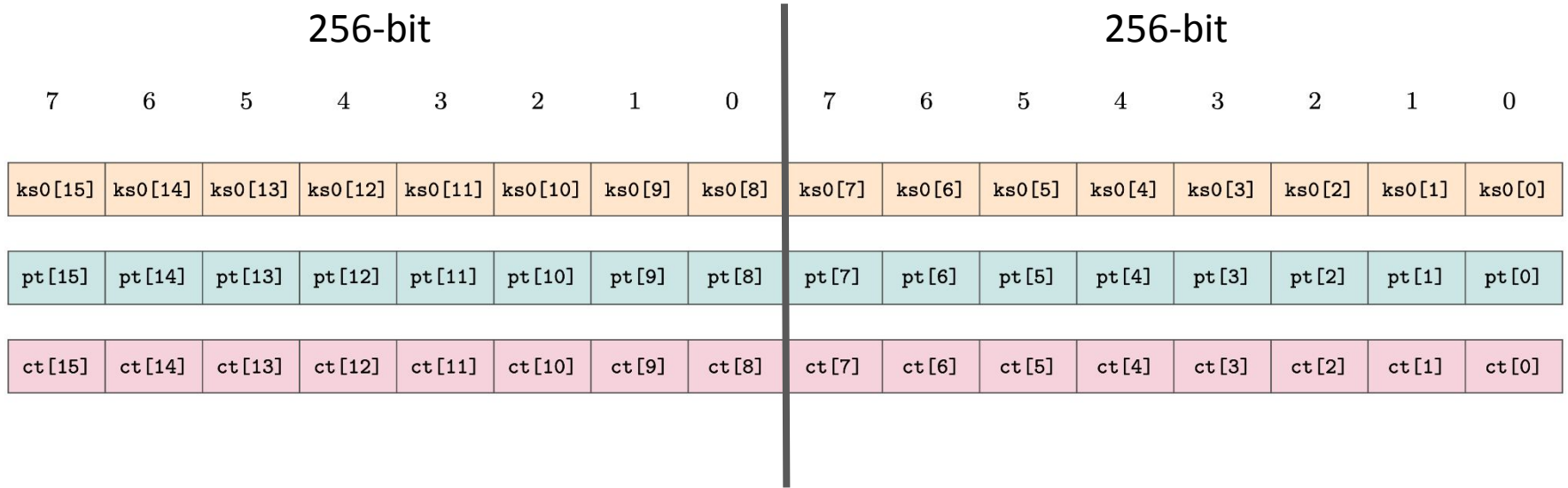
Chacha Vectorization (Idea)

	7	6	5	4	3	2	1	0
ymm0	S0	S0	S0	S0	S0	S0	S0	S0
ymm1	S1	S1	S1	S1	S1	S1	S1	S1
ymm2	S2	S2	S2	S2	S2	S2	S2	S2
ymm3	S3	S3	S3	S3	S3	S3	S3	S3
ymm4	k[0]	k[0]	k[0]	k[0]	k[0]	k[0]	k[0]	k[0]
ymm5	k[1]	k[1]	k[1]	k[1]	k[1]	k[1]	k[1]	k[1]
ymm6	k[2]	k[2]	k[2]	k[2]	k[2]	k[2]	k[2]	k[2]
ymm7	k[3]	k[3]	k[3]	k[3]	k[3]	k[3]	k[3]	k[3]
ymm8	k[4]	k[4]	k[4]	k[4]	k[4]	k[4]	k[4]	k[4]
ymm9	k[5]	k[5]	k[5]	k[5]	k[5]	k[5]	k[5]	k[5]
ymm10	k[6]	k[6]	k[6]	k[6]	k[6]	k[6]	k[6]	k[6]
ymm11	k[7]	k[7]	k[7]	k[7]	k[7]	k[7]	k[7]	k[7]
ymm12	ctr+0	ctr+1	ctr+2	ctr+3	ctr+4	ctr+5	ctr+6	ctr+7
ymm13	n[0]	n[0]	n[0]	n[0]	n[0]	n[0]	n[0]	n[0]
ymm14	n[1]	n[1]	n[1]	n[1]	n[1]	n[1]	n[1]	n[1]
ymm15	n[2]	n[2]	n[2]	n[2]	n[2]	n[2]	n[2]	n[2]



	7	6	5	4	3	2	1	0
ymm0	k7[0]	k6[0]	k5[0]	k4[0]	k3[0]	k2[0]	k1[0]	k0[0]
ymm1	k7[1]	k6[1]	k5[1]	k4[1]	k3[1]	k2[1]	k1[1]	k0[1]
ymm2	k7[2]	k6[2]	k5[2]	k4[2]	k3[2]	k2[2]	k1[2]	k0[2]
ymm3	k7[3]	k6[3]	k5[3]	k4[3]	k3[3]	k2[3]	k1[3]	k0[3]
ymm4	k7[4]	k6[4]	k5[4]	k4[4]	k3[4]	k2[4]	k1[4]	k0[4]
ymm5	k7[5]	k6[5]	k5[5]	k4[5]	k3[5]	k2[5]	k1[5]	k0[5]
ymm6	k7[6]	k6[6]	k5[6]	k4[6]	k3[6]	k2[6]	k1[6]	k0[6]
ymm7	k7[7]	k6[7]	k5[7]	k4[7]	k3[7]	k2[7]	k1[7]	k0[7]
ymm8	k7[8]	k6[8]	k5[8]	k4[8]	k3[8]	k2[8]	k1[8]	k0[8]
ymm9	k7[9]	k6[9]	k5[9]	k4[9]	k3[9]	k2[9]	k1[9]	k0[9]
ymm10	k7[10]	k6[10]	k5[10]	k4[10]	k3[10]	k2[10]	k1[10]	k0[10]
ymm11	k7[11]	k6[11]	k5[11]	k4[11]	k3[11]	k2[11]	k1[11]	k0[11]
ymm12	k7[12]	k6[12]	k5[12]	k4[12]	k3[12]	k2[12]	k1[12]	k0[12]
ymm13	k7[13]	k6[13]	k5[13]	k4[13]	k3[13]	k2[13]	k1[13]	k0[13]
ymm14	k7[14]	k6[14]	k5[14]	k4[14]	k3[14]	k2[14]	k1[14]	k0[14]
ymm15	k7[15]	k6[15]	k5[15]	k4[15]	k3[15]	k2[15]	k1[15]	k0[15]

Chaha Vectorization (Idea cont.)



For each plaintext block:

1. split plaintext and extracted keystream in 2 vectors
2. XOR plaintext with extracted keystreams

Vectorization Implementation

1. Store working chacha states in vectors
2. Perform vectorized QUARTER_ROUND
3. Engineer left rotation from other instructions

- speedup 6.01x (-O3), 3.99x (-O0)

```
#define ROTL32_256(x, n) \
    _mm256_or_si256(_mm256_slli_epi32(x, n), _mm256_srli_epi32(x, 32 - n))

#define QUARTER_ROUND_256(a, b, c, d) \
( \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = ROTL32_256(d, 16), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 12), \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = ROTL32_256(d, 8), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 7) \
)
```

Quarter Round improvement - Vectorization

- Trick for rotations by 8 and 16 using 1 instruction

Effect: reduced instruction count

- speedup: ~6.8x (-O3), ~2.21x (-O0)

```
#define ROT16_MASK _mm256_set_epi8(\
    13,12,15,14, 9,8,11,10, 5,4,7,6, 1,0,3,2,\
    13,12,15,14, 9,8,11,10, 5,4,7,6, 1,0,3,2)

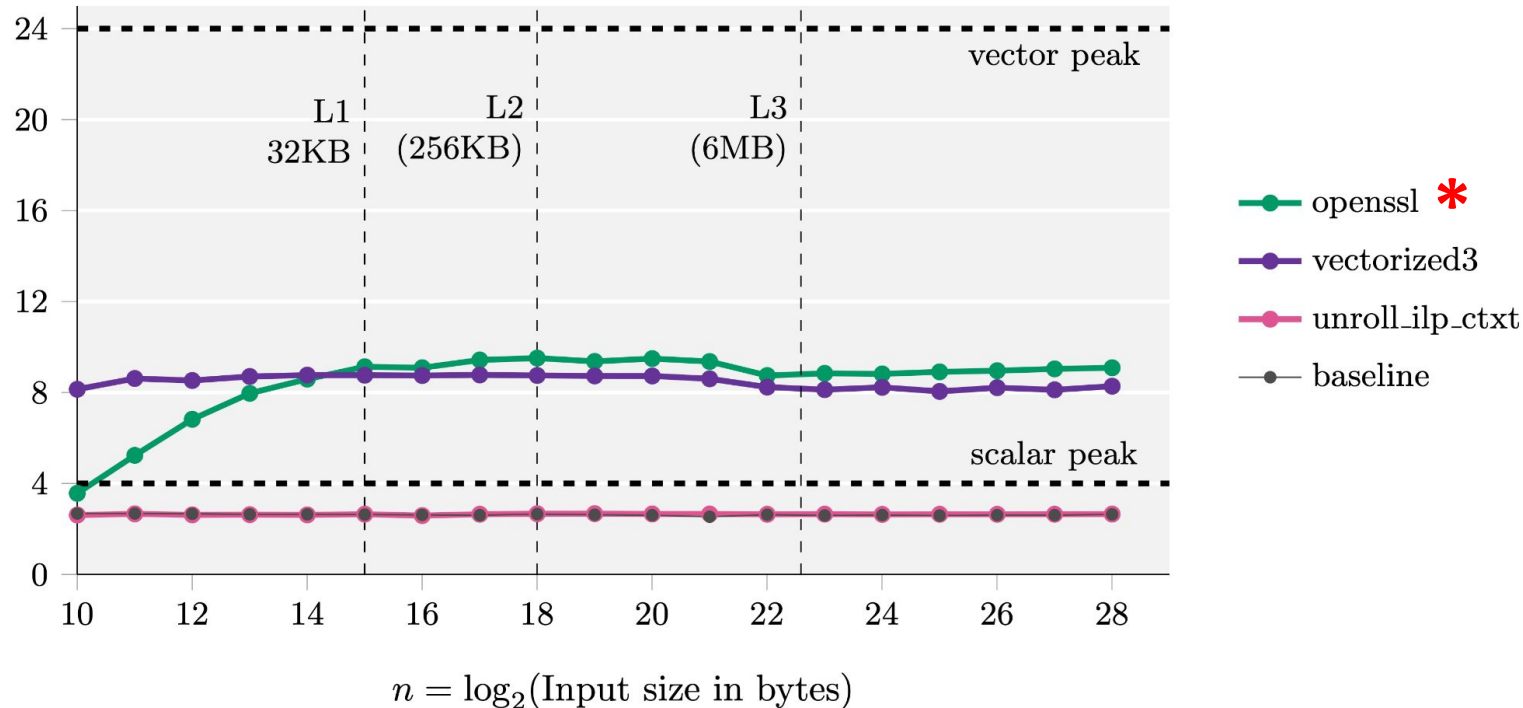
#define ROT8_MASK _mm256_set_epi8(\
    14,13,12,15, 10,9,8,11, 6,5,4,7, 2,1,0,3,\
    14,13,12,15, 10,9,8,11, 6,5,4,7, 2,1,0,3)
```

```
#define QUARTER_ROUND_256_2(a, b, c, d) \
( \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = _mm256_shuffle_epi8(d, ROT16_MASK), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 12), \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = _mm256_shuffle_epi8(d, ROT8_MASK), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 7) \
)
```

Performance of Chacha Encrypt, Intel Core i7-7700

Compiler: GCC 13.3.0, Flags: -O3 -march=native

ops/cycle



*

Openssl op count **assumed** same as of our vectorized version

Poly1305 setup

- **r** and **acc** as 5 limbs of 32 bits:
 - **unsaturated** (useful for carry delay)
- **s** left as a variable of 4 limbs of 32 bits
- Output:
 - 16 B tag (polynomial hash evaluation of ciphertext)

Profiling: Poly1305 create_tag (Runtime)

- mul_mod_p() → **37%**
- to_large_num_rep1305() → **36%**
- add_large_nums55() → 12%

- **Bottlenecks:** to_large_num_rep1305() , mul_mod_p()

- Source: valgrind

To_large_num_representation1305()

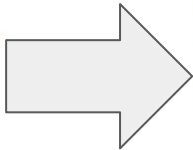
1. Reduce op count by unroll & ILP
2. Better variable allocation

```
void to_large_num_rep(uint32_t out[5], const unsigned c
uint64_t t[5] = {0};

for (uint64_t i = 0; i < len_bytes; i++){
    t[i/4] |= ((uint64_t)bytes[i] << ((i%4)*8));
}

out[0] = (uint32_t)(t[0]) & mask_lowest_26bits;
int left_shift = 6;
int right_shift = 26;
for(int i = 0; i < 4; i++){
    out[i+1] = (uint32_t)((t[i] >> right_shift) |
    (t[i+1] << left_shift)) & mask_lowest_26bits;

    left_shift += 6;
    right_shift -= 6;
}
}
```



```
//to_large_num_rep(n_0, block_0, 17);
uint64_t low = *(const uint64_t*)&block_0[0];
uint64_t high = *(const uint64_t*)&block_0[8];
uint64_t last = block_0[16];
n_0[0] = (uint32_t)(low) & 0x3FFFFFFF;
n_0[1] = (uint32_t)(low >> 26) & 0x3FFFFFFF;

n_0[2] = (uint32_t)((low >> 52) | (high << 12)) & 0x3FFFFFFF;

n_0[3] = (uint32_t)(high >> 14) & 0x3FFFFFFF;

n_0[4] = (uint32_t)((high >> 40) | (last << 24)) & 0x3FFFFFFF;
```

Poly1305: parallel_Horner_create_tag()

Algorithm 1 Poly1305 Parallel Horner

```
1:  $acc_1, acc_2, acc_3, acc_4 \leftarrow 0$ 
2: for each  $(M_1, M_2, M_3, M_4)$  in data do
3:    $acc_1 \leftarrow acc_1 \cdot r^4 + M_1$ 
4:    $acc_2 \leftarrow acc_2 \cdot r^4 + M_2$ 
5:    $acc_3 \leftarrow acc_3 \cdot r^4 + M_3$ 
6:    $acc_4 \leftarrow acc_4 \cdot r^4 + M_4$ 
7: end for
8:  $acc \leftarrow acc_1 \cdot r^4 + acc_2 \cdot r^3 + acc_3 \cdot r^2 + acc_4 \cdot r$ 
9: for  $i = 1$  to  $(n \bmod 4)$  do
10:   $acc \leftarrow (acc + M_{4\lfloor n/4 \rfloor + i}) \cdot r$ 
11: end for
```

- Precomputation of r^4
- Alignment of accumulators after main loop
- Speedup: 3.1x (-O3)

Poly1305: carry_delay

- Precomputation of r^4 and r^8
- Reducing ops(1 carry propagation every 2 multiplications)
- 8 multiplications but only 4 accumulators
- Speedup: 4.7x (-O3)

Algorithm 2 Poly1305 2-carry-delay with parallel Horner

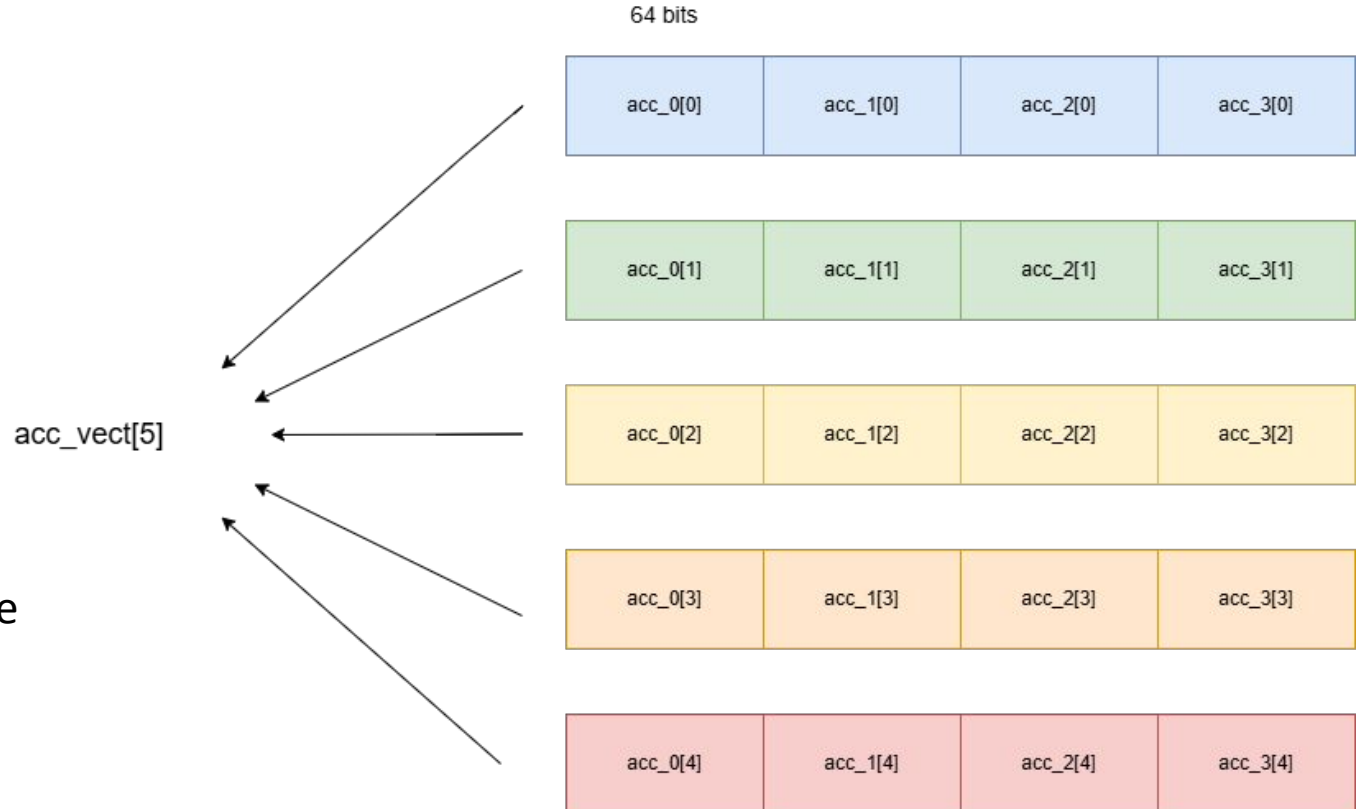
```
1:  $acc_1, acc_2, acc_3, acc_4 \leftarrow 0$ 
2: for each  $(M_1, M_2, M_3, M_4, M_5, M_6, M_7, M_8)$  in data do
3:    $acc_1 \leftarrow acc_1 \cdot r^8 + M_1 \cdot r^4 + M_5$ 
4:    $acc_2 \leftarrow acc_2 \cdot r^8 + M_2 \cdot r^4 + M_6$ 
5:    $acc_3 \leftarrow acc_3 \cdot r^8 + M_3 \cdot r^4 + M_7$ 
6:    $acc_4 \leftarrow acc_4 \cdot r^8 + M_4 \cdot r^4 + M_8$ 
7:   Carry propagation and reduction
8: end for
9:  $acc \leftarrow acc_1 \cdot r^4 + acc_2 \cdot r^3 + acc_3 \cdot r^2 + acc_4 \cdot r$ 
10: for  $i = 1$  to  $(n \bmod 8)$  do
11:    $acc \leftarrow (acc + M_{8\lfloor n/8 \rfloor + i}) \cdot r$ 
12: end for
```

Poly1305: vectorization of carry delay

Variables vectorized in
carry_delay:

- accumulators
- data blocks
- r^4 and r^8

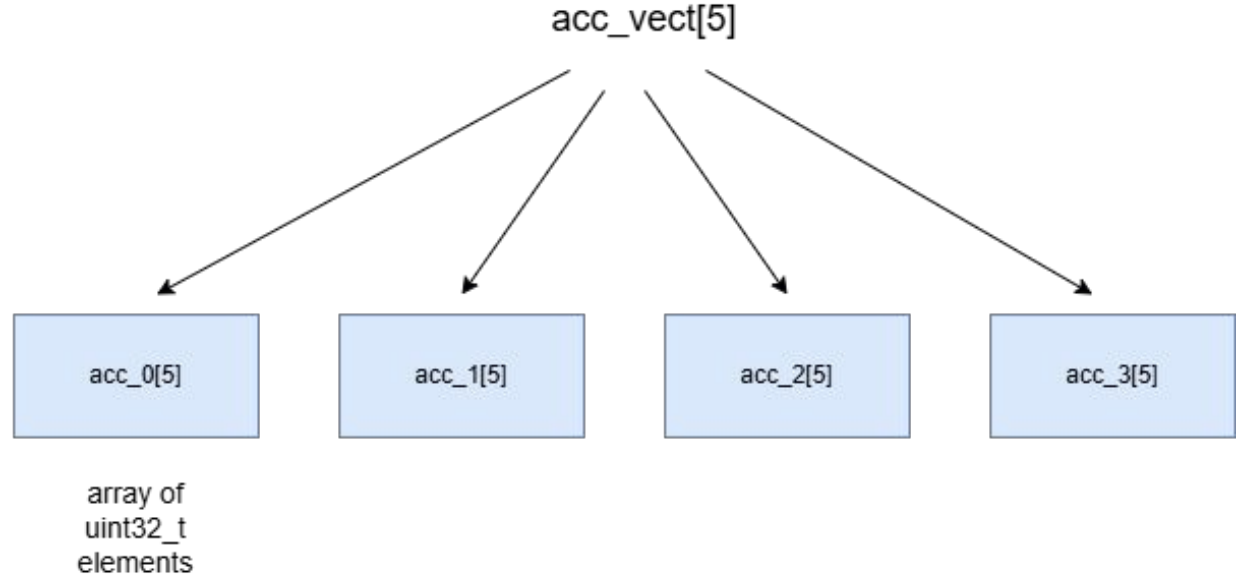
Variables become
arrays of 5 vectors, one
for each limb



Poly1305: vectorization of carry delay

- `acc_vect` is unpacked for alignment of accumulators

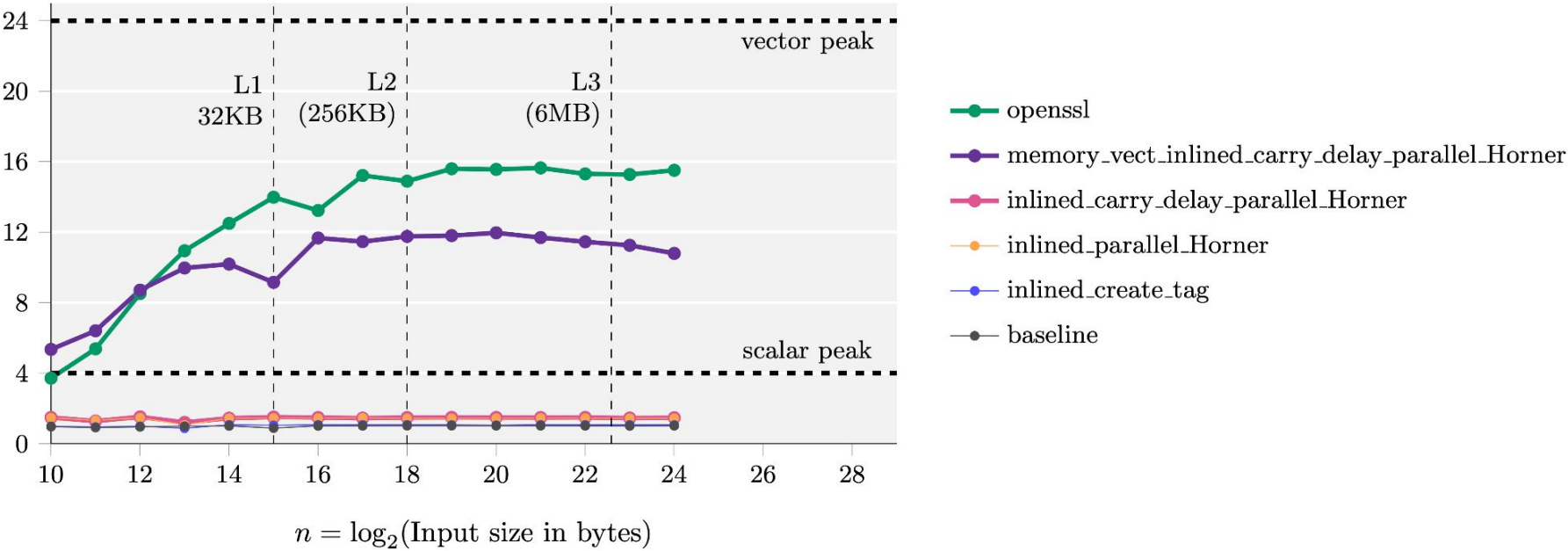
- Speedup: 13x (-O3)



Performance of Poly1305 Create Tag, Intel Core i7-7700HQ

Compiler: GCC 13.3.0, Flags: -O3 -fno-tree-vectorize -march=native

ops/cycle



Poly2133: Profiling Results (Runtime)

Same bottlenecks as Poly1305

- 67% of runtime in mulmod_p()
specifically multiplication loop ->
- 23% to_large_num_rep() convert to 8 limb representation
- rest < 10%
- Valgrind tool = callgrind with message_size > L1 cache

```
// ----- mulmod_p(acc, r) -----  
uint64_t mult_small[NUM_LIMBS] = {0};  
uint64_t mult_large[NUM_LIMBS] = {0};  
uint64_t mask_lowest_6bits = 0x3f;  
  
for (int i = 0; i < NUM_LIMBS; i++){  
    for(int j = 0; j < NUM_LIMBS; j++){  
        int k = i+j;  
        if (k < NUM_LIMBS){  
            mult_small[k] += (uint64_t)acc[i]* r[j];  
        }  
        else{  
            mult_large[k - NUM_LIMBS] += (uint64_t)acc[i]* 3*r[j];  
        }  
    }  
}
```

Poly2133 Implementation Choices

8x32 bit limbs -> **unsaturated, unbalanced 7x28 + 17 bits**

-> **4 bits headroom** for delayed carry

-> **32 bits** because **omit** using **uint128_t** in multiplication part

message block size = 26 bytes

-> **largest possible still in primefield**, so **reduce mulmod calls** (which is overhead)

tag size = 26 byte

-> largest possible still in primefield, truncates like poly1305

Poly2133: Scalar Optimizations

- **inline** all function calls -> ~1.1x speedup (with -O0)
-> **eliminates** function call **overhead**
- **unroll** loops to add **ILP, precomputations & less memory access** -> ~1.5x
- **2-level** approach (**4 blocks**) + **delayed carry** -> ~4.2x
-> can work on 4 independent blocks in **parallel**

Poly2133: Vectorization

- idea: vectorize over accumulators (in multiplication loop)

```
mult_small[i+j] += n1_plus_acc* r4[j] + (uint64_t)n2[i]* r3[j]  
                + (uint64_t)n3[i]* r2[j] + (uint64_t)n4[i]* r[j];
```



define:

vec_n =

(n1 + acc)[i]	n2[i]	n3[i]	n4[i]
---------------	-------	-------	-------

vec_r =

r ⁴ [j]	r ³ [j]	r ² [j]	r[j]
--------------------	--------------------	--------------------	------

mult_small[i+j] = mult(vec_n, vec_r)

Poly2133: Vectorization ~5.8x

idea: do **all computations for one limb**, immediately **store result**, **save carry** and **move on** to next limb

-> could go up to **8 parallel accumulators** instead of 4 without register spilling



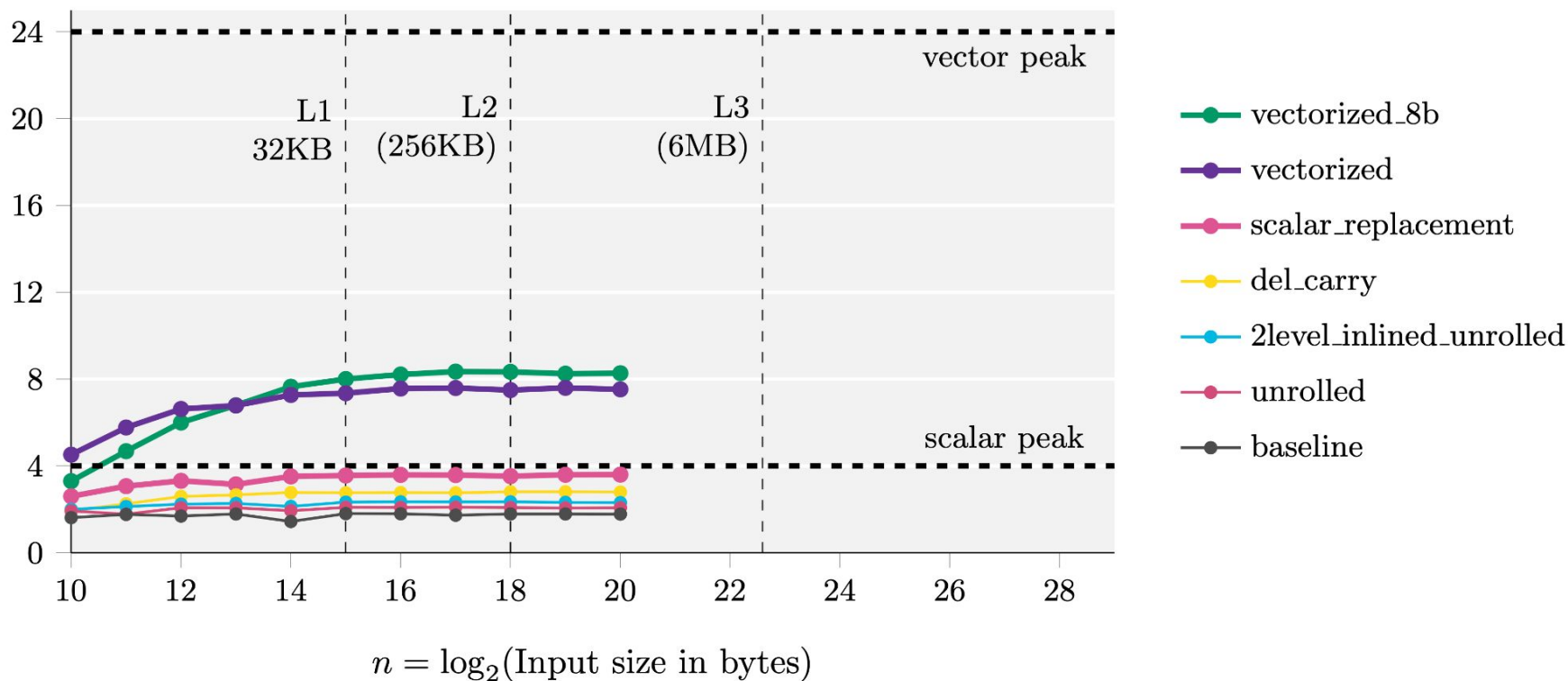
```
/* mult_large computes _m1 = (n1+acc)*3*r^4 + 3*n2*r^3 + 3*n3*r^2 + 3*n4*r) (for all (i,j) pairs for limb0 mult_large)*/ \
_vl = _mm256_mul_epu32(_mm256_set_epi64x(_d1, _c1, _b1, _a1), _mm256_set_epi64x((three_r)[7],(three_r2)[7],(three_r3)[7],(three_r4)[7])); \
_vl = _mm256_add_epi64(_vl, _mm256_mul_epu32(_mm256_set_epi64x(_d2, _c2, _b2, _a2), _mm256_set_epi64x((three_r)[6],(three_r2)[6],(three_r3)[6],(three_r4)[6]))); \
_vl = _mm256_add_epi64(_vl, _mm256_mul_epu32(_mm256_set_epi64x(_d3, _c3, _b3, _a3), _mm256_set_epi64x((three_r)[5],(three_r2)[5],(three_r3)[5],(three_r4)[5]))); \
_vl = _mm256_add_epi64(_vl, _mm256_mul_epu32(_mm256_set_epi64x(_d4, _c4, _b4, _a4), _mm256_set_epi64x((three_r)[4],(three_r2)[4],(three_r3)[4],(three_r4)[4]))); \
_vl = _mm256_add_epi64(_vl, _mm256_mul_epu32(_mm256_set_epi64x(_d5, _c5, _b5, _a5), _mm256_set_epi64x((three_r)[3],(three_r2)[3],(three_r3)[3],(three_r4)[3]))); \
_vl = _mm256_add_epi64(_vl, _mm256_mul_epu32(_mm256_set_epi64x(_d6, _c6, _b6, _a6), _mm256_set_epi64x((three_r)[2],(three_r2)[2],(three_r3)[2],(three_r4)[2]))); \
_vl = _mm256_add_epi64(_vl, _mm256_mul_epu32(_mm256_set_epi64x(_d7, _c7, _b7, _a7), _mm256_set_epi64x((three_r)[1],(three_r2)[1],(three_r3)[1],(three_r4)[1]))); \
_low=_mm256_castsi256_si128(_vl); _high=_mm256_extracti128_si256(_vl,1); \
_sum_vec=_mm_add_epi64(_low,_high); \
_m1=(uint64_t)_mm_extract_epi64(_sum_vec,0)+(uint64_t)_mm_extract_epi64(_sum_vec,1); \
_sum=_ms+((_m1 & mask_lowest_17bits)<<11)+_wrap_carry+_carry; \
(acc)[0]=(uint32_t)(_sum & mask_lowest_28bits); \
_carry=_sum>>28; _wrap_carry=_m1>>17; \
```

Limitation: - **repeated set_epi64x** calls because only use temporaries
- **serial addition chain** (from _vl accumulator)
- **reductions after limbs** expensive (add up all entries)

Performance of Poly2133 Create Tag, Intel Core i7-7700HQ

Compiler: GCC 13.3.0, Flags: -O3 -march=native

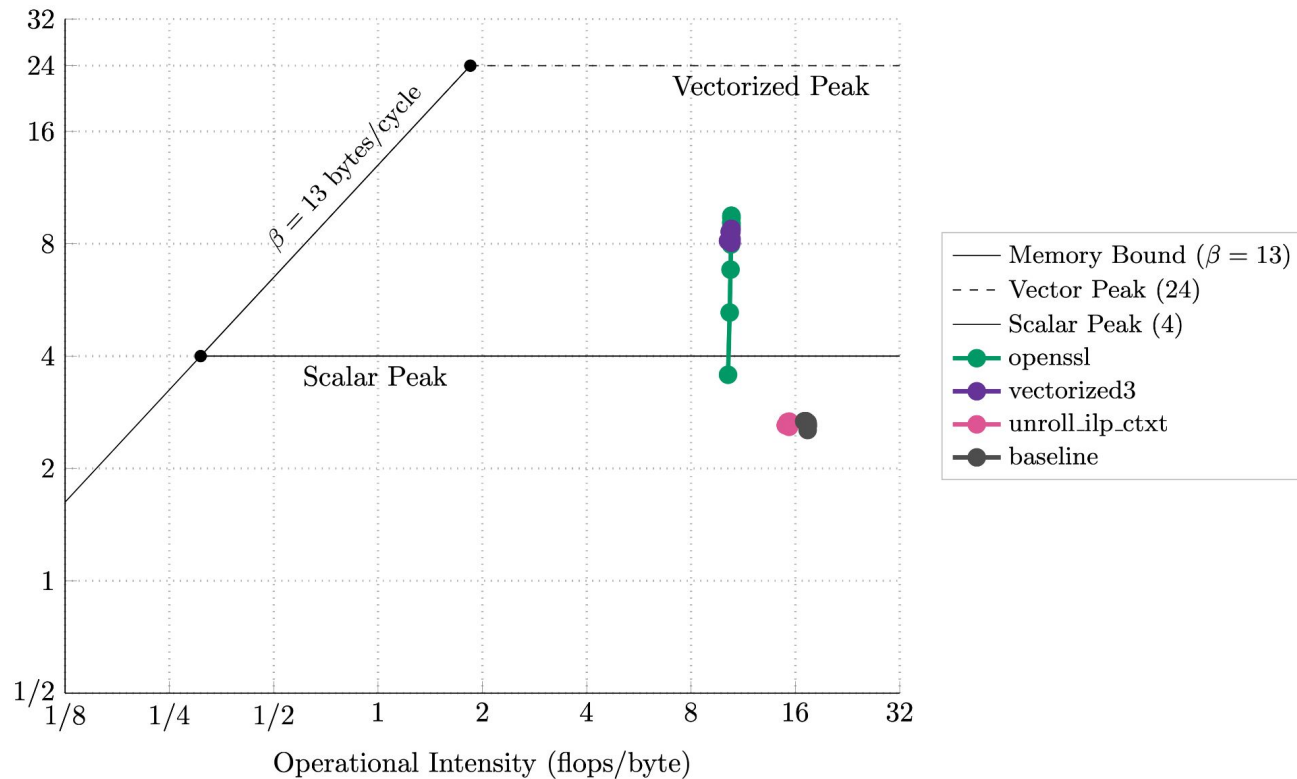
ops/cycle



Roofline Plot: ChaCha20 Encrypt, Intel Core i7-7700HQ

Compiler: GCC 13.3.0, Flags: -O3 -march=native

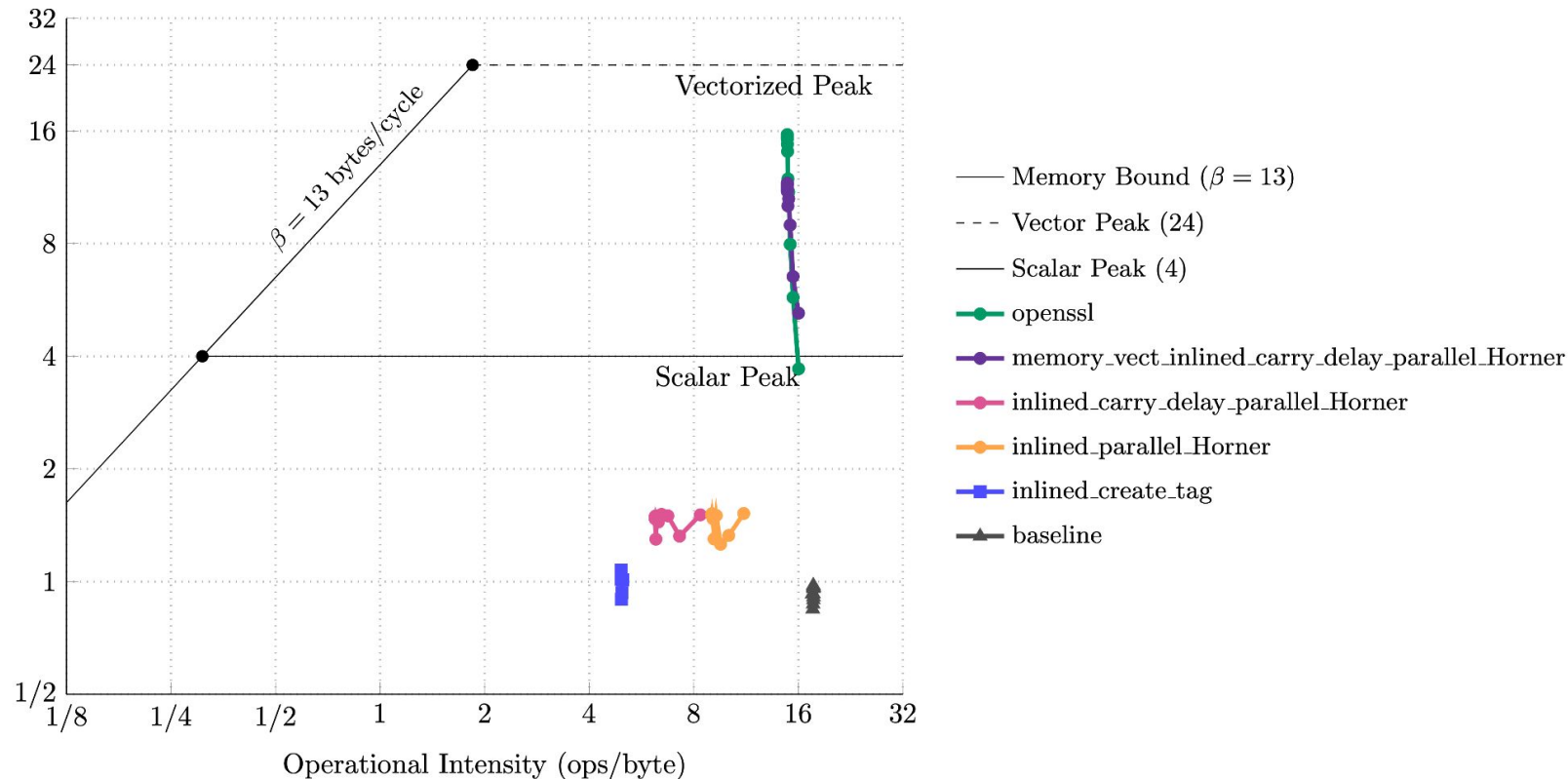
Performance (ops/cycle)



Roofline Plot: Poly1305 Create Tag, Intel Core i7-7700HQ

Compiler: GCC 13.3.0, Flags: -O3 -march=native

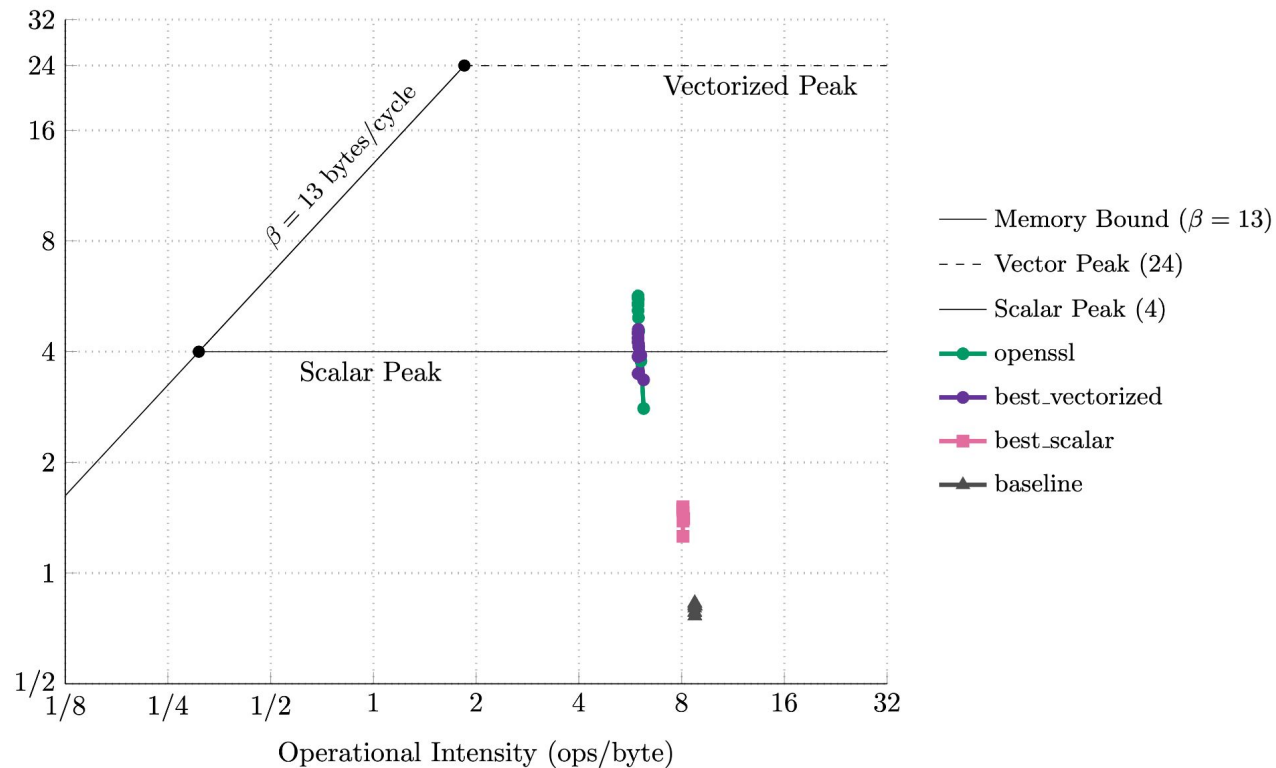
Performance (ops/cycle)



Roofline Plot: AEAD1305 Encrypt, Intel Core i7-7700HQ

Compiler: GCC 13.3.0, Flags: -O3 -march=native

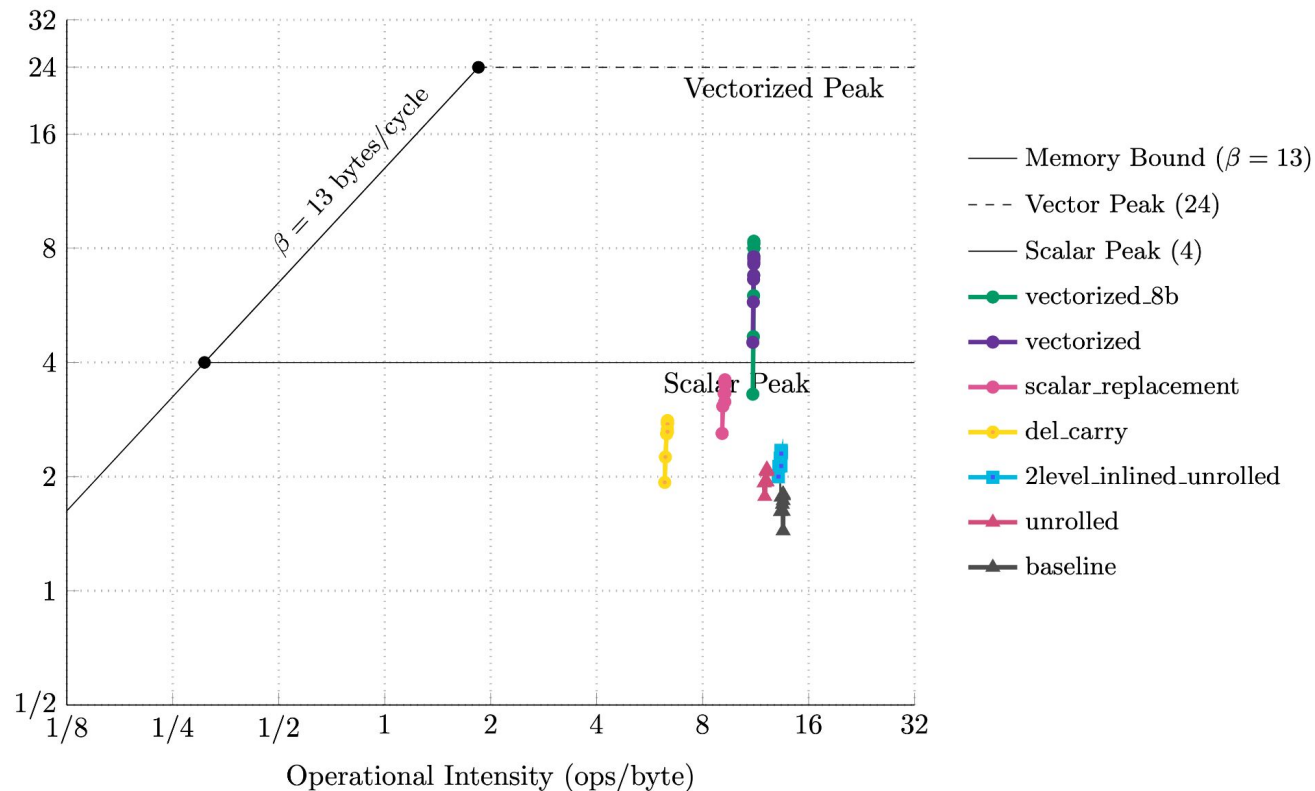
Performance (ops/cycle)



Roofline Plot: Poly2133 Create Tag, Intel Core i7-7700HQ

Compiler: GCC 13.3.0, Flags: -O3 -march=native

Performance (ops/cycle)



Thank you for your attention!

Appendix

ChaCha20 - Strength Reduction

- Strength reduction (**1 div, 1 mod**);
- XOR 64 bit words, rather than byte by byte
- speedup approx. 1.17x (-O3), 1.09x (-O0)
- Reasoning:
 - At O0, block computation dominates, so strength reduction is only small improvement
- **Assumption**: The host machine is little endian

ChaCha20 - Inlining

- Inline all helper functions and state initialization
- Macros for reusable elements
- Speedup of approx. 1.46x (-O3), 2.04x (-O0)
- Reasoning:
 - reduces overhead of function calls,
 - allows compiler to optimize better (ILP, out of order execution)
 - Quarter rounds are the overhead
 - Dependency in quarter rounds happens every 4 rounds

```
#define ROTL32(v, n) \
    ( (v << (n)) | (v >> (32 - (n))) )

#define QUARTER_ROUND(a, b, c, d) \
( \
    a += b, \
    d ^= a, \
    d = ROTL32(d,16), \
    c += d, \
    b ^= c, \
    b = ROTL32(b,12), \
    a += b, \
    d ^= a, \
    d = ROTL32(d,8), \
    c += d, \
    b ^= c, \
    b = ROTL32(b,7) \
)
```

Chacha20 - Scalar Replacement

- Store working state in scalar variables to keep in registers
- Speedup of approx. 1.52x (-O3), 2.23x (-O0)
- Reasoning:
 - Avoid reads and writes for every double round -> reduced memory traffic

```
for (uint64_t b = 0; b < num_full_blocks; b++) {  
    o0 = state[0];    o1 = state[1];    o2 = state[2];    o3 = state[3];  
    o4 = state[4];    o5 = state[5];    o6 = state[6];    o7 = state[7];  
    o8 = state[8];    o9 = state[9];    o10 = state[10];   o11 = state[11];  
    o12 = state[12];  o13 = state[13];  o14 = state[14];  o15 = state[15];  
  
    for (int i = 0; i < DOUBLE_ROUNDS; i++) {  
        QUARTER_ROUND(o0, o4, o8, o12);  
        QUARTER_ROUND(o1, o5, o9, o13);  
        QUARTER_ROUND(o2, o6, o10, o14);  
        QUARTER_ROUND(o3, o7, o11, o15);  
  
        QUARTER_ROUND(o0, o5, o10, o15);  
        QUARTER_ROUND(o1, o6, o11, o12);  
        QUARTER_ROUND(o2, o7, o8, o13);  
        QUARTER_ROUND(o3, o4, o9, o14);  
    }  
}
```

Chacha Vectorization (Implementation)

- Store working state in vectors
- Perform vectorized QUARTER_ROUND
- **16 x 256bit registers => 8 full ChaCha states**
=> Handle 8 states at once
- speedup: ~2.41x (-O3), ~2.95x (-O0)
- Reasoning:
 - Cleanly allows handling of multiple plaintext blocks
 - Extracting the values out of the vectors and storing into the vectors is an overhead

```
#define ROTL32_256(x, n) \
    _mm256_or_si256(_mm256_slli_epi32(x, n), _mm256_srli_epi32(x, 32 - n))

#define QUARTER_ROUND_256(a, b, c, d) \
( \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = ROTL32_256(d, 16), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 12), \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = ROTL32_256(d, 8), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 7) \
)
```

Second Vectorization

- Store plaintext in vector
- Transform state vectors to fit 1 state in 2 vectors
- do XOR with plaintext using vector instr.
- speedup 6.01x (-O3), 3.99x (-O0)
- Reasoning:
 - States already in registers -> can do more computations with them without much overhead
 - Extraction: double 8x8 vector transposition -> i-th keystream stored as vector

Third Vectorization

- new Quarter_round with reduced instruction count
- Leverages artificial, **but faster** left rotation when rotation multiple of 8
- Uses vector mask
- speedup: ~6.8x (-O3), ~2.21x (-O0)
- Reasoning:
 - The shuffle mask needs to be set up in the registers
 - O3 optimized the setup of the mask and the ordering of instructions

```
#define ROT16_MASK _mm256_set_epi8(\
    13,12,15,14, 9,8,11,10, 5,4,7,6, 1,0,3,2,\
    13,12,15,14, 9,8,11,10, 5,4,7,6, 1,0,3,2)\
\
#define ROT8_MASK _mm256_set_epi8(\
    14,13,12,15, 10,9,8,11, 6,5,4,7, 2,1,0,3,\
    14,13,12,15, 10,9,8,11, 6,5,4,7, 2,1,0,3)
```

```
#define QUARTER_ROUND_256_2(a, b, c, d) \
(\
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = _mm256_shuffle_epi8(d, ROT16_MASK), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 12), \
    a = _mm256_add_epi32(a, b), \
    d = _mm256_xor_si256(d, a), \
    d = _mm256_shuffle_epi8(d, ROT8_MASK), \
    c = _mm256_add_epi32(c, d), \
    b = _mm256_xor_si256(b, c), \
    b = ROTL32_256(b, 7) \
)
```

Poly2133 Speedups at one glance

Implementation	-O3 -fno-vec		-O0 -fno-vec		-O3	
	Cycles	Speedup	Cycles	Speedup	Cycles	Speedup
inlined	1,225,060	1.10x	1,309,640	1.12x	1,115,060	1.11x
unrolled	930,385	1.45x	970,903	1.51x	874,582	1.41x
2level_basic	1,261,370	1.07x	1,405,320	1.04x	1,188,740	1.04x
2level_inl_unr	857,084	1.57x	1,037,290	1.41x	904,901	1.36x
delcarry	364,886	3.69x	350,354	4.18x	341,982	3.61x
precomp	411,100	3.28x	401,075	3.65x	534,986	2.31x
remif	384,277	3.51x	429,181	3.41x	402,960	3.06x
scalrep	414,168	3.25x	427,442	3.43x	401,934	3.07x
vec	234,125	5.76x	239,453	6.12x	239,294	5.15x
vec_8b	203,772	6.61x	220,856	6.63x	212,428	5.81x

Table 2: Poly2133 `create_tag` performance: cycles and speedup over baseline.
mean over 50 runs with 50'000 bytes message size

Optimization overview for Poly2133

- Bottlenecks that were taken into consideration and optimized:
 - Bottleneck of Poly2133 is mulmod_p function (67% of runtime), specifically the multiplication loop and to_large_num_rep function (23% of runtime) where we convert bytes to 64-bit integers.
- Create_tag:
 - inlined all functions (so help compiler optimize)
 - unrolled all carry independent loops (reduces computations and accesses to memory)
 - 2-level approach from paper with 4 blocks on top of unrolled and inlined (so computations for messages can happen more in parallel and we have less sequential dependencies)
 - added precomputations (i.e. $3*r \rightarrow$ three_r computed once before the message block and acc+n in multiplication loop) (reduces computations)
 - replaced the if/else in the multiplication loop through for loops (better for vectorization)
 - added scalar replacement in mulmod_p function $\rightarrow$ the whole multiplication loop got unrolled and now we processed limb for limb and instantly did carries, so we could minimize the number of variables in registers.
 - experimented with different compiler optimizations -O0, to -O3
 -
- Init: inlining, unrolling, vectorization

Poly2133 Implementation Choices

8x32 bit limbs -> unsaturated, unbalanced $7 \times 28 + 17$ bits

- > 4 bits headroom for delayed carry

- > 8 limbs for vectorization

- > as balanced as possible with most even numbers

- > 32 bits because omit using `uint128_t` in multiplication part

message block size = 26 bytes

- > largest possible still in primefield, so reduce `mulmod` calls (which is overhead)

tag size = 26 byte

- > largest possible still in primefield, truncates like `poly1305`

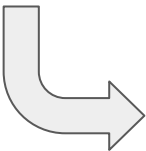
Poly2133: Profiling Results (Runtime: to_large_num_rep)

```
// ----- to_large_num_rep(r, r_bytes, half_key_len) -----  
uint64_t tr[NUM_LIMBS] = {0};  
  
for (int i = 0; i < half_key_len; i++){  
    tr[i/4] |= ((uint64_t)r_bytes[i] << ((i%4)*8));  
}  
  
for(int i = 0; i < 7; i++){  
    int bit_start = 28*i;  
    int t_idx = bit_start / 32;  
    int bit_in_t = bit_start % 32;  
    r[i] = (uint32_t)((tr[t_idx] >> bit_in_t) | (tr[t_idx + 1] << (32 - bit_in_t))) & mask_lowest_28bits;  
}  
  
r[7] = (uint32_t)(tr[6] >> 4) & mask_lowest_17bits;
```

Poly2133: vectorization of to_large_num_rep failed

- tried vectorizing this runtime heavy loop, but no improvement
- Reason: awkward extraction and storing because of length 27
- We opted to unroll everything so compiler can help as much as possible in this case

```
uint64_t _tr[LIMBS_2133] = {0}; \
for (uint64_t _i = 0; _i < (len); _i++) { \
    _tr[_i/4] |= ((uint64_t)(bytes)[_i] << ((_i%4)*8)); \
} \
```



```
_m128i _bytes16_lo = _mm_loadu_si128((__m128i*)(bytes)); \
_m128i _bytes16_hi = _mm_loadu_si128((__m128i*)((bytes) + 11)); \
_m128i _bytes16_hi_shifted = _mm_srli_si128(_bytes16_hi, 1); \
\
_m256i _tr_low = _mm256_cvtepu32_epi64(_bytes16_lo); \
_m256i _tr_high = _mm256_cvtepu32_epi64(_bytes16_hi_shifted); \
\
uint64_t _tr[LIMBS_2133] = {0}; \
_mm256_storeu_si256((__m256i*)&_tr[0], _tr_low); \
_mm256_storeu_si256((__m256i*)&_tr[3], _tr_high); \
```

Poly2133: Optimizations (inline)

- inlined all function calls
- no compiler optimizations (so comparable)
- ~1.1x speedup with -O0
 - > eliminates function call overhead

Poly2133: Optimizations (unroll + ILP + precompute) ~1.5x

Get speedup because:

- results in less ops
- less memory access

```
for (int i = 0; i < half_key_len; i++){
    tr[i/4] |= ((uint64_t)r_bytes[i] << ((i%4)*8));
}

for(int i = 0; i < 7; i++){
    int bit_start = 28*i;
    int t_idx = bit_start / 32;
    int bit_in_t = bit_start % 32;
    r[i] = (uint32_t)((tr[t_idx] >> bit_in_t) | (tr[t_idx + 1] <
```



```
for (int i = 0; i+4 < half_key_len; i+= 4){
    uint64_t word = (uint64_t)r_bytes[i]
        | ((uint64_t)r_bytes[i + 1] << 8)
        | ((uint64_t)r_bytes[i + 2] << 16)
        | ((uint64_t)r_bytes[i + 3] << 24);
    tr[i/4] = word;
}

tr[6] = (uint64_t)r_bytes[24]
    | ((uint64_t)r_bytes[25] << 8)
    | ((uint64_t)r_bytes[26] << 16);

r[0] = (uint32_t)(tr[0] | (tr[1] << 32)) & mask_lowest_28bits;
r[1] = (uint32_t)((tr[0] >> 28) | (tr[1] << 4)) & mask_lowest_28bits;
r[2] = (uint32_t)((tr[1] >> 24) | (tr[2] << 8)) & mask_lowest_28bits;
r[3] = (uint32_t)((tr[2] >> 20) | (tr[3] << 12)) & mask_lowest_28bits;
r[4] = (uint32_t)((tr[3] >> 16) | (tr[4] << 16)) & mask_lowest_28bits;
r[5] = (uint32_t)((tr[4] >> 12) | (tr[5] << 20)) & mask_lowest_28bits;
r[6] = (uint32_t)((tr[5] >> 8) | (tr[6] << 24)) & mask_lowest_28bits;
r[7] = (uint32_t)(tr[6] >> 4) & mask_lowest_17bits;
```

Poly2133: Optimizations (2-level approach)

- 4 message blocks in parallel

instead of: $((((n1*r) + n2)*r + n3)*r + n4)*r$

we do: $n1*r^4 + n2*r^3 + n3*r^2 + n4*r$

- worse speedup because does 5 mulmod_p per 4 blocks, not only 4

```
to_large_num_rep(n, block, block_len + 1);  
add_large_nums_88(acc, n); // acc += n  
mulmod_p(acc, r); // acc = (acc * r) mod p
```



```
to_large_num_rep(n1, block1, BLOCK_SIZE + 1);  
to_large_num_rep(n2, block2, BLOCK_SIZE + 1);  
to_large_num_rep(n3, block3, BLOCK_SIZE + 1);  
to_large_num_rep(n4, block4, BLOCK_SIZE + 1);
```

```
mulmod_p(n1, r4); // n1 = n1 * r^4 mod p  
mulmod_p(n2, r3); // n2 = n2 * r^3 mod p  
mulmod_p(n3, r2); // n3 = n3 * r^2 mod p  
mulmod_p(n4, r); // n4 = n4 * r mod p
```

```
add_large_nums_88(n1, n2); // n1 += n2  
add_large_nums_88(n1, n3); // n1 += n3  
add_large_nums_88(n1, n4); // n1 += n4
```

```
mulmod_p(acc, r4); // acc = acc * r^4 mod p  
add_large_nums_88(acc, n1); // acc += n1
```

speedup: ~1.4x

Poly2133: Optimizations (delayed carry) ~4.2x

- instead of 5x mulmod, repack & normalize (for each of 4 blocks + acc)
 - > 1x mulmod, repack & normalize

5x

```
for (int i = 0; i < NUM_LIMBS; i++){  
    for(int j = 0; j < NUM_LIMBS; j++){  
        int k = i+j;  
        if (k < NUM_LIMBS){  
            mult_small[k] += (uint64_t)acc[i]* r[j];  
        }  
        else{  
            mult_large[k - NUM_LIMBS] += (uint64_t)acc[i]* 3*r[j];  
        }  
    }  
}
```



1x

```
for (int i = 0; i < NUM_LIMBS; i++){  
    for(int j = 0; j < NUM_LIMBS; j++){  
        int k = i+j;  
        if (k < NUM_LIMBS){  
            mult_small[k] += (uint64_t)n1[i]* r4[j] + (uint64_t)n2[i]* r3[j]  
                + (uint64_t)n3[i]* r2[j] + (uint64_t)n4[i]* r[j]  
                + (uint64_t)acc[i]* r4[j];  
        }  
        else{  
            mult_large[k - NUM_LIMBS] += (uint64_t)n1[i]* 3*r4[j] + (uint64_t)n2[i]* 3*r3[j]  
                + (uint64_t)n3[i]* 3*r2[j] + (uint64_t)n4[i]* 3*r[j]  
                + (uint64_t)acc[i]* 3*r4[j];  
        }  
    }  
}
```

Poly2133: failed optimizations

- **precompute $3*r^k$** and store in three_rk arrays:
 - > too much **memory overhead**
- **replace if/else** in multiplication loop:
 - > **branch prediction** of compiler **better, double loop overhead**
& built on top of precomputation
- **scalar replacement:**
 - > **too many scalars** for compiler to juggle
 - > but **used structure for vectorization** to **circumvent register spillings**

Poly2133: Optimizations (pre-computations) ~3.7x

- precompute $3*r^k$ outside of message loop ($3*r \rightarrow \text{three_r}$)
- gets worse than delayed carry for both -O0 and -O3 $\rightarrow$ probably because more memory accesses and compiler can vectorize better, respectively

```
mult_large[k - NUM_LIMBS] += (uint64_t)n1[i]* 3*r4[j] + (uint64_t)n2[i]* 3*r3[j]
                             + (uint64_t)n3[i]* 3*r2[j] + (uint64_t)n4[i]* 3*r[j]
                             + (uint64_t)acc[i]* 3*r4[j];
```

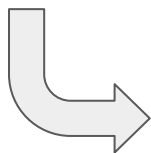


```
mult_large[k - NUM_LIMBS] += (uint64_t)n1[i]* three_r4[j] + (uint64_t)n2[i]* three_r3[j]  
+ (uint64_t)n3[i]* three_r2[j] + (uint64_t)n4[i]* three_r[j]  
+ (uint64_t)acc[i]* three_r4[j];
```

Poly2133: Optimizations (remove if/else) ~3.4x

```
for (int i = 0; i < NUM_LIMBS; i++){
    for(int j = 0; j < NUM_LIMBS; j++){
        int k = i+j;
        if (k < NUM_LIMBS){
            mult_small[k] += (uint64_t)n1[i]* r4[j] + (uint64_t)n2[i]* r3[j]
                           + (uint64_t)n3[i]* r2[j] + (uint64_t)n4[i]* r[j]
                           + (uint64_t)acc[i]* r4[j];
        }
        else{
            mult_large[k - NUM_LIMBS] += (uint64_t)n1[i]* three_r4[j] + (uint64_t)n2[i]* three_r3[j]
                                         + (uint64_t)n3[i]* three_r2[j] + (uint64_t)n4[i]* three_r[j]
                                         + (uint64_t)acc[i]* three_r4[j];
        }
    }
}
```

- worse than delayed carry, maybe because branch easily predictable & now double loop overhead & still uses three_r
- **thought helps vectorization later**
- **precompute $n1[i] + acc[i]$**



```
for (int i = 0; i < NUM_LIMBS; i++){
    uint64_t n1_plus_acc = (uint64_t)(n1[i] + acc[i]);
    int k = 0;
    for (int j = 0; j < NUM_LIMBS - i; j++){
        mult_small[i+j] += n1_plus_acc* r4[j] + (uint64_t)n2[i]* r3[j]
                        + (uint64_t)n3[i]* r2[j] + (uint64_t)n4[i]* r[j];
    }
    for (int j = NUM_LIMBS - i; j < NUM_LIMBS; j++){
        mult_large[k] += n1_plus_acc* three_r4[j] + (uint64_t)n2[i]* three_r3[j]
                      + (uint64_t)n3[i]* three_r2[j] + (uint64_t)n4[i]* three_r[j];
        k++;
    }
}
```

Poly2133: Optimizations (scalar replacement) $\sim 3.4x$

```
for (int i = 0; i < NUM_LIMBS; i++){
    uint64_t n1_plus_acc = (uint64_t)(n1[i] + acc[i]);
    int k = 0;
    for (int j = 0; j < NUM_LIMBS - i; j++){
        mult_small[i+j] += n1_plus_acc* r4[j] + (uint64_t)n1[i] * r3[j];
        mult_small[i+j] += (uint64_t)n1[i] * r2[j] + (uint64_t)n4[i]* r[j];
    }
    for (int j = NUM_LIMBS - i; j < NUM_LIMBS; j++){
        mult_large[k] += n1_plus_acc* three_r4[j] + (uint64_t)n2[i]* three_r3[j]
        + (uint64_t)n3[i]* three_r2[j] + (uint64_t)n4[i]* three_r[j];
        k++;
    }
}
```

```
uint64_t _carry = 0; \
uint64_t _wrap_carry = 0; \
uint64_t _sum; \
for(int _i = 0; _i < 7; _i++){ \
    _sum = _mult_small[_i] + (((_mult_large[_i] & mask_lowest_17bits) \
    (acc)[_i] = (uint32_t)(_sum & mask_lowest_28bits); \
    _carry = _sum >> 28; \
    _wrap_carry = (_wrap_carry + _carry) >> 17; \
} \
\
_sum = _mult_small[_i] + (((_mult_large[_i] & mask_lowest_17bits) \
(acc)[_i] = (uint32_t)(_sum & mask_lowest_28bits); \
_carry = _sum >> 17; \
_wrap_carry = (_wrap_carry + _carry) >> 6; \
\
uint64_t _total_carry = (_carry + _wrap_carry) * 3; \
uint64_t _sum0 = (uint64_t)(acc)[0] + _total_carry; \
(acc)[0] = (uint32_t)(_sum0 & mask_lowest_28bits); \
_carry = _sum0 >> 28; \
(acc)[1] += (uint32_t)_carry; \
_carry = (acc)[1] >> 28; \
(acc)[1] &= mask_lowest_28bits; \
(acc)[2] += (uint32_t)_carry; \
```

1. Multiplication loop

store everything to acc & handle carries

8x
(for each limb)

- only with restructuring get similar speedup as previous optimization with -O3 (because memory traffic + serial dependency of summation)

```
/* compute mult_small and mult_large for limb 0 and immediately store in acc[0] and handle carry
_ms = _a0*(r4)[0] + _b0*(r3)[0] + _c0*(r2)[0] + _d0*(r)[0]; \
_m1 = _a1*(three_r4)[7] + _b1*(three_r3)[7] + _c1*(three_r2)[7] + _d1*(three_r)[7] \
+ _a2*(three_r4)[6] + _b2*(three_r3)[6] + _c2*(three_r2)[6] + _d2*(three_r)[6] \
+ _a3*(three_r4)[5] + _b3*(three_r3)[5] + _c3*(three_r2)[5] + _d3*(three_r)[5] \
+ _a4*(three_r4)[4] + _b4*(three_r3)[4] + _c4*(three_r2)[4] + _d4*(three_r)[4] \
+ _a5*(three_r4)[3] + _b5*(three_r3)[3] + _c5*(three_r2)[3] + _d5*(three_r)[3] \
+ _a6*(three_r4)[2] + _b6*(three_r3)[2] + _c6*(three_r2)[2] + _d6*(three_r)[2] \
+ _a7*(three_r4)[1] + _b7*(three_r3)[1] + _c7*(three_r2)[1] + _d7*(three_r)[1]; \
_sum = _ms + ((_m1 & mask_lowest_17bits) << 11) + _wrap_carry + _carry; \
(acc)[0] = (uint32_t)(_sum & mask_lowest_28bits); \
_carry = _sum >> 28; _wrap_carry = _m1 >> 17; \
```

-> now compute all sums for one limb, store to acc immediately and extract carries for next limb, go on to next

Poly2133: Optimizations (naive vectorization)

-> is worse than scalar version

Reason:

- have 16x256-bit registers
- would need $5 \times 8 = 40$ registers
-> register spilling

```
__m256i vec_n[NUM_LIMBS]; \
__m256i vec_r[NUM_LIMBS]; \
__m256i vec_three_r[NUM_LIMBS]; \
__m256i vec_mult_small[NUM_LIMBS]; \
__m256i vec_mult_large[NUM_LIMBS]; \
for (int _i = 0; _i < NUM_LIMBS; _i++) { \
    vec_mult_small[_i] = _mm256_setzero_si256(); \
    vec_mult_large[_i] = _mm256_setzero_si256(); \
    vec_r[_i] = _mm256_set_epi64x((r)[_i], (r2)[_i], (r3)[_i], (r4)[_i]); \
    vec_three_r[_i] = _mm256_set_epi64x((three_r)[_i], (three_r2)[_i], (three_r3)[_i]); \
    vec_n[_i] = _mm256_set_epi64x((uint64_t)(n4)[_i], (uint64_t)(n3)[_i], (uint64_t)(n2)[_i], (uint64_t)(n1)[_i]); \
} \
for (int _i = 0; _i < NUM_LIMBS; _i++){ \
    \
    for (int _j = 0; _j < NUM_LIMBS - _i; _j++){ \
        (vec_mult_small)[_i+_j] = _mm256_add_epi64((vec_mult_small)[_i+_j], (vec_r)[_i]); \
    } \
    for (int _j = NUM_LIMBS - _i; _j < NUM_LIMBS; _j++){ \
        (vec_mult_large)[_i+_j - NUM_LIMBS] = _mm256_add_epi64((vec_mult_small)[_j - NUM_LIMBS], (vec_r)[_i]); \
    } \
} \
```

5x8 registers needed!

Poly2133: Why 8 blocks during vectorization?

- get diminishing returns from adding parallel blocks
 - can see in plots how the difference between 4 to 8 is already not too large
 - Trade-off between #reductions per block and #scalars compiler needs to load:
 - 1 Block: 16 reductions/block & 8 scalars
 - 4 Blocks: $16/4 = 4$ reductions/block & $4*8 = 32$ scalars (+ 12 reduction gain, but + 24 scalars)
 - 8 Blocks: $16/8 = 2$ reductions/block & $8*8 = 64$ scalars (+ 2 reduction gain, but + 32 scalars)
 - 12 Blocks: $16/12 = 1.3$ reductions/block & $12*8 = 96$ scalars (+ 0.7 red. gain, but + 32 scalars)
- > for 8 Blocks still reasonable gain, even though scalars expensive
- similar diminishing returns for carry-propagation (but less important)

Why mod $2^{130}-5 = *5$

- in prime field $p = 2^{130}-5$ the number $p = 0$

$$\rightarrow 2^{130}-5 = 0 \text{ (in prime field)}$$

$$\rightarrow 2^{130} = 5$$

so if we have an entry in limb corresponding to 2^{130} we can take this entry, multiply by 5 and add to the first limb

ChaCha20 - Unrolling + ILP

- Unroll by 8 + ILP in ciphertext computation
 - no significant speedup but also not many operations on top of the strength reduction
- Why?
 - Each ciphertext block independent from the other
 - 4 ALU ports -> aiming for full throughput
 - Reduce number of for loop increments
- Speedup:
 - 1.50x with -O3, 2.38x with -O0 (4 blocks at once)
 - 1.27x with -O3, 2.38x with -O0 (8 blocks at once)
- Reasoning (Speedup):
 - Register Spilling, O3 does a lot on its own and gets backed into corner
 - Ilp is maximized already with 4 blocks (with -O0)

Functions Called

- (1) `ChaCha_20_block(key, nonce) → poly_key`
- (2) `ChaCha_20_encrypt(ptxt, key, nonce, block_ctr) → ctxt`
- (3) `Poly1305_init(poly_key) → r, s`
- (4) `Create_tag(mac_data, r, s) → tag`

```
mac_data = pad16(aad) | pad16(ctxt) | aad_len_{64} |  
           ctxt_len_{64}
```

Cost Analysis on our first implemented functions:

$$\begin{aligned}(1)+(2)+(3)+(4) &= O(1962) + O(n/64 * 2221) + O(200) + O((n+m)/16 * 271) \\ &= O(n) \text{ (considering } n \gg m)\end{aligned}$$

Poly2133: Profiling Results (Cache)

Valgrind tool = cachegrind with message_size > 4*L3 cache:

- instructions stay in cache
- only have significant slowdown for loading message if message_size > L3
- > **~100% of L1 cache misses are from memcpy (loads message)**
- **working set stays in cache** since very small (~224 bytes)

Variable	Limbs	Limb size	Total
acc	8	4 bytes	32 bytes
r	8	4 bytes	32 bytes
mult_small	8	8 bytes	64 bytes
mult_large	8	8 bytes	64 bytes
n	8	4 bytes	32 bytes
Total			≈ 224 bytes